

## Task 4: Evaluation of the Network Architecture Model

### A. Provide a brief explanation of how the student should approach the evaluation process.

To evaluate the model, the student should review the training and validation results using accuracy as the primary metric; F1 score, precision, and recall per class to identify any weaknesses; and loss to identify overfitting or underfitting. The student must determine if the model is learning and generalizing effectively by comparing these results to the baseline and any previous iterations.

The student must then compare the confusion matrices and performance metrics with the baseline and previous iterations to determine if there are meaningful improvements or justifiable tradeoffs. The student should use a mix of confusion matrices and line graphs, along with any other visualizations deemed necessary, to both review the results and provide the information to the instructor for review. Based on this information, the student must choose the best-performing model and evaluate it against the test set.

After successful testing, the student must use the CIFAR-10-C dataset to evaluate the robustness of the model. There is no threshold for these results. Instead, the student should use the results to determine how well the model generalizes to corrupted and transformed data.

The student must then provide a final report explaining the model's components, performance, limitations, issues, possible biases, real-world uses, and other topics outlined in Part I.

### B. Discuss the impact of using stopping criteria instead of defining the number of epochs.

Stopping criteria is a powerful tool for managing overfitting and training time. When using stopping criteria with a set patience parameter, the model will stop training when accuracy fails to improve after a specified number of epochs (Keras, 2026a). If epochs are defined, the model continues to train regardless of improvement, potentially leading to overfitting and/or unnecessary computation.

For the final model, the epochs were set to 50, monitoring validation accuracy with a stopping patience of 10. In addition, a ReduceLROnPlateau callback monitoring loss was added with a patience of 3. This addition adjusts the learning rate when validation loss stops improving, allowing for small weight adjustments that can improve performance (Keras, 2026b).

The model stopped training on epoch 32, meaning that the weights of epoch 22 were restored and used as the result. Epoch 22 returned an accuracy value of 89.2%.

#### 1. Include a screenshot showing the final training epoch.

Final epoch ran:

```
Epoch 32: val_accuracy did not improve from 0.89210
157/157 — 78s 494ms/step — accuracy: 0.9895 — loss: 0.0319 — precision: 0.9904 — recall: 0.9887 — val_accuracy: 0.8905
V4 model saved to best_model_v4_final.keras
```

Full text of epoch:

```
Epoch 32: val_accuracy did not improve from 0.89210
157/157 — 78s 494ms/step — accuracy: 0.9895 — loss: 0.0319 —
precision: 0.9904 — recall: 0.9887 — val_accuracy: 0.8905 — val_loss: 0.4832 —
val_precision: 0.8960 — val_recall: 0.8868 — learning_rate: 6.2302e-06
```

Final epoch saved:

```
Epoch 22/50
157/157 — 0s 468ms/step — accuracy: 0.9874 — loss: 0.0387 — precision: 0.9886 — recall: 0.9860
Epoch 22: ReduceLROnPlateau reducing learning rate to 4.9841302825370803e-05.

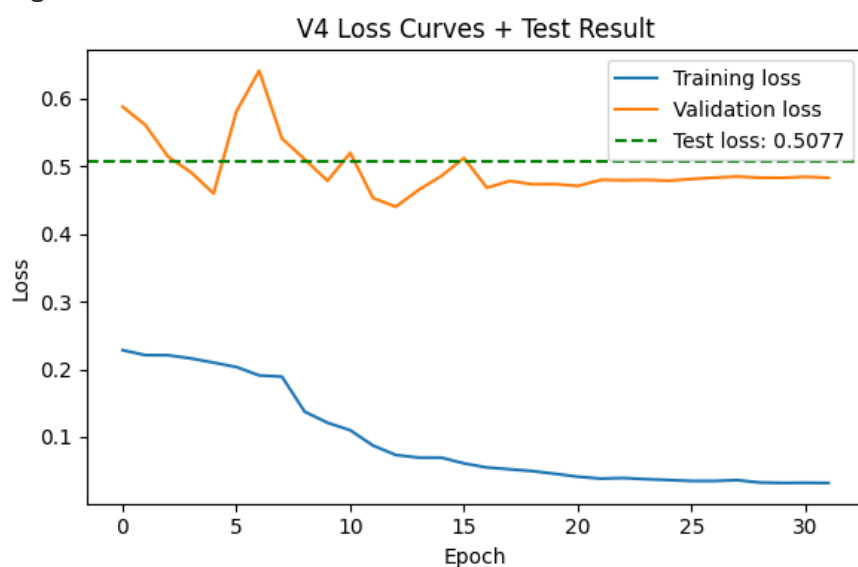
Epoch 22: val_accuracy improved from 0.89170 to 0.89210, saving model to best_model_v4_weights.keras

Epoch 22: finished saving model to best_model_v4_weights.keras
```

C. Include following three types of visualizations to provide a comprehensive view of the model's performance:

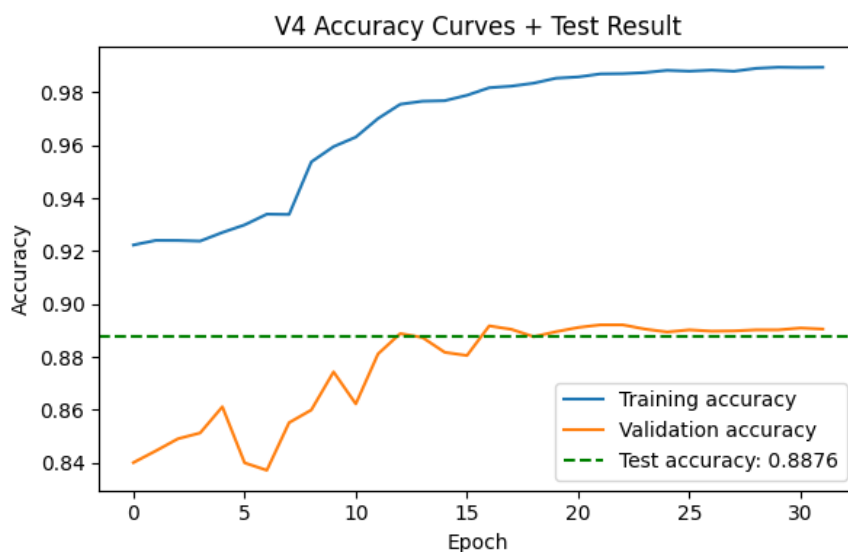
- loss curves

Figure 1: Version 4 Loss Curves



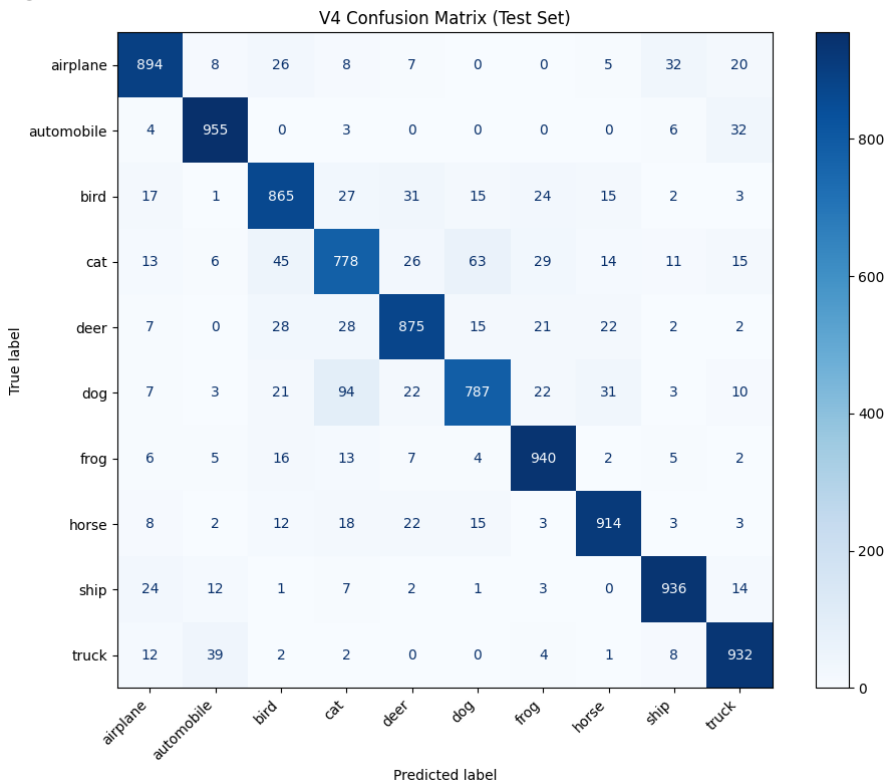
- accuracy curves

Figure 2: Version 4 Accuracy Curves



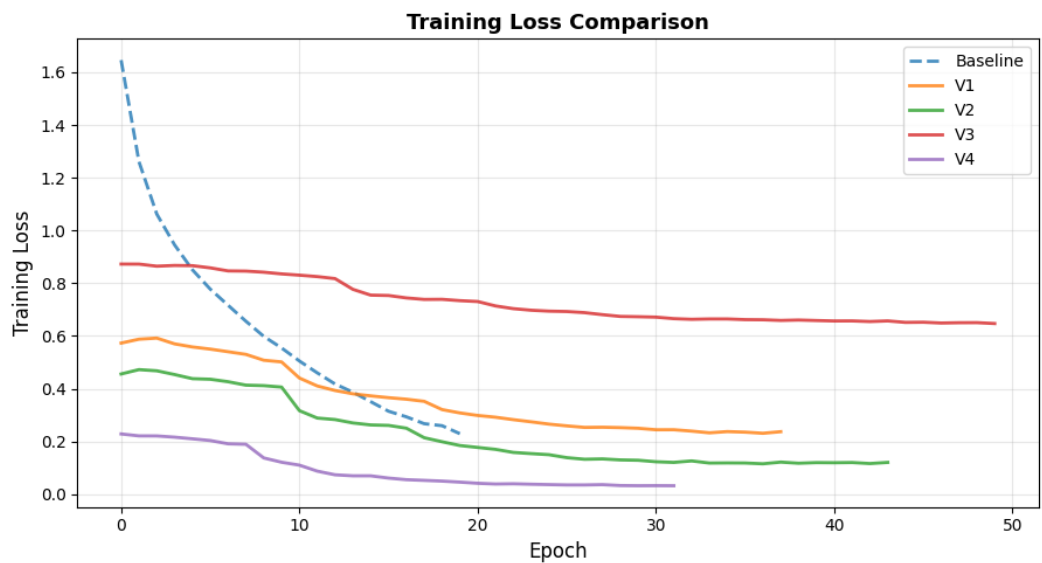
- confusion matrices

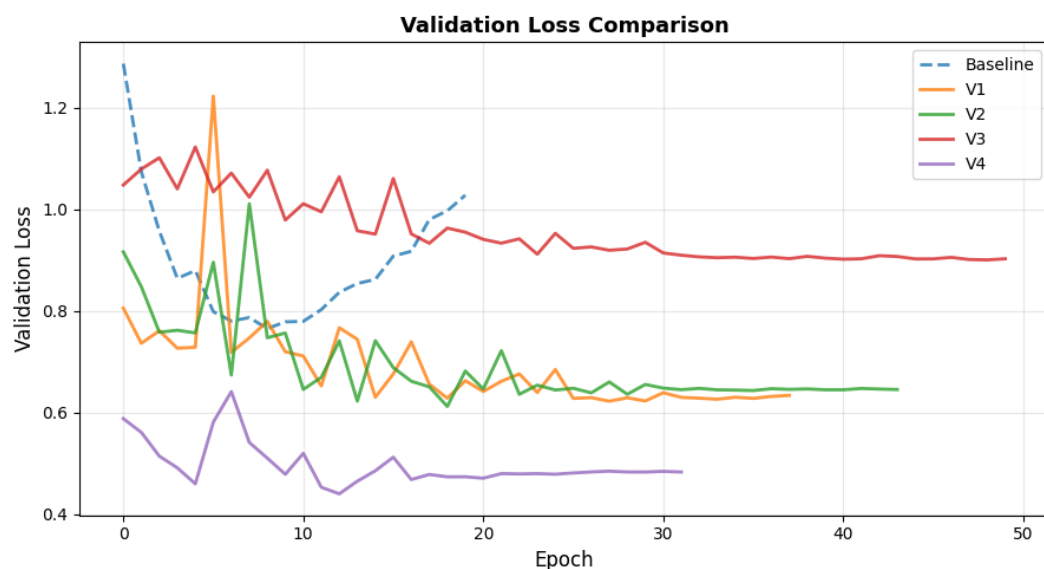
Figure 3: Version 4 Confusion Matrix (Test Set)



D. Discuss the measures that were taken to address overfitting, including an explanation of how model complexity and dataset characteristics influence overfitting.

Figure 4: Training Loss Comparison



**Figure 5: Validation Loss Comparison**

Several measures were taken to address overfitting during the development of the model. First, from the baseline model on, a dropout layer was implemented to randomly deactivate neurons during training, which prevents overfitting by removing nodes the model may be too reliant on for prediction (Keras, 2026c). This layer was preserved throughout the development process with rates between 20-50% tested using Bayesian optimization.

This was the only overfitting safeguard implemented on the baseline model, and the validation loss (shown in Figure 5) was capable of being reduced below .8 before rising back to 1.02 due to a lack of early stopping criteria. With this in mind, early stopping was implemented alongside a ReduceLROnPlateau callback. The stopping criteria monitored validation accuracy with a patience of 10, while learning rate was decreased by 50% when loss plateaued for three epochs. This, in tandem with hyperparameter tuning using Bayesian optimization, reduced the loss of the model significantly in subsequent versions (apart from Version 3 which is detailed below).

In addition to this technique, data augmentation using random flip, rotation, zoom, and contrast was used to boost the diversity of the dataset, encouraging the model to generalize and reduce overfitting to images (particularly with a small dataset like CIFAR-10, explained below). Batch normalization was also added to provide regularization and improve convergence by stabilizing activations throughout the network. In combination with ReLU activation, it also helps to mitigate vanishing gradient issues (Gupta, 2021)

I decided to try and apply two methods to Version 3 that are regularly used to reduce overfitting: L2 regularization and model smoothing. L2 regularization penalizes the model for large weights, forcing the model to keep weights as small as possible. These simpler representations can boost the generalization of the model and reduce overfitting (Nagpal, 2024). Label smoothing adds regularization by distributing a small amount of probability from the correct class to the incorrect classes, which in many cases can reduce overfitting by forcing the model to reduce its proclivity to seek maximum confidence (Geeks for Geeks, 2025a). Surprisingly this resulted in the model taking a major step back, with lower accuracy and F1 scores, and higher training loss and validation loss. This implied that Version 2 was already regularized sufficiently and the additional constraints detrimentally affected the model's ability to learn.

I investigated other methods for improving model performance and decided to try implementing a visual geometry group (VGG)-type architecture, which operates on the principle that two smaller convolutional layers prior to pooling provide better learning results than a single convolutional layer (Vipin, 2024). This model version provided a significant improvement at the cost of increased training time and computational demand (16 hours, 43 minutes of training; eight convolutional layers instead of four).

The use of VGG-inspired architecture highlights the importance of model complexity when addressing overfitting. As shown in the Figures 3 and 4, above, Version 4 performed significantly better with eight convolutional layers as opposed to four. More layers give the model the opportunity to learn richer features but require the use of regularization techniques to reduce the risk of overfitting. With the measures we had in place, we were able to both reduce overfitting and improve accuracy. Without the regularization techniques, this model would be more prone to overfitting, as the additional layers would support memorization over generalization.

The final model presented a gap of approximately 10% between training and validation, which indicates overfitting is still present. However, validation loss decreased substantially, with .483 validation loss compared to Version 1's .633, while accuracy, F1 score, precision, and recall improved across the board, with accuracy in particular rising by close to 5%. Most telling is that test accuracy and loss were very close to the validation accuracy, showing that the model did not overfit to the validation set significantly and can reproduce results on newly introduced data. This shows that the model is generalizing well and the regularization techniques are effective, finding a good balance between memorization and abstraction.

Additionally, it is important to note the role that the dataset itself has in overfitting. CIFAR-10 is composed of 50,000 training images that are 32x32 pixels and classified into 10 categories. This number is further reduced when applying a 20% validation split for hyperparameter tuning. Because the dataset is relatively small compared to sets used for modern computer vision models, the model has fewer training samples to learn from, increasing the risk of overfitting (Geeks for Geeks, 2025b). To combat this, I used data augmentation to increase variation. However, even with this addition, it is fair to say the small dataset contributed to the final overfitting we see in the model.

### **E. Justify the choice of the primary evaluation metric based on project goals and dataset properties.**

The primary evaluation metric used for this project was validation accuracy. This was due to several factors. First, the CIFAR-10 dataset is a balanced dataset. Because we do not have to account for class imbalance, accuracy is a reliable indicator of model performance. Second, the classification objective treats all classes equally, meaning that the ability to classify a plane is equally as important as classifying a cat, and misclassifying any class is equally as problematic as any other. As accuracy inherently treats all classes equally in determining the performance of the model, the metric aligns well with the goal of the assignment. Finally, using validation accuracy shows how the model is performing against unseen data, providing a better indication of performance than training accuracy.

However, accuracy on its own is not enough to ensure an informative metric. In tandem with accuracy, the model's F1 score, precision, and recall per class were monitored for any class-specific biases or weaknesses. This is essential for verifying that accuracy represents a reliable metric. For example, with 10 classes equally balanced and an accuracy metric of 80%, it is possible that one or more classes are contributing to the errors far more than the others. A model performing poorly on specific classes represents issues that need addressing, and pairing these metrics with accuracy ensures that performance is improving across classes rather than being driven by a subset.

## F. Describe in detail the data augmentation techniques used and their impact on model performance.

Data augmentation was used in training to improve model robustness and generalization. Additionally, the CIFAR-10-C dataset was used to evaluate the final model to assess overall robustness.

In training, I used RandomFlip, RandomRotation, RandomZoom, and RandomContrast from the Keras SDK to improve the performance of the model.

RandomFlip randomly flips images horizontally with a 50% probability. The inclusion of this layer increases the training data without additional storage and tests the model for its ability to identify objects when mirrored.

RandomRotation randomly rotates images by up to approximately  $\pm 36^\circ$ . This layer encourages the model to adapt to real-world situations where images are captured at varying angles. It also helps curb model memorization of image features based on orientation.

RandomZoom randomly zooms in or out of images by 10%, which helps the model learn by passing images at varying sizes, improving robustness by challenging the model to correctly classify objects captured at different distances.

RandomContrast randomly adjusts the contrast of images by  $\pm 10\%$ . This helps the model train for real-world image challenges such as lighting conditions and improves the overall robustness of the model by introducing variations in lighting conditions and image quality.

Overall, these transformations work together to increase the diversity of training data, expose the model to variations that encourage generalization, and improve performance on unseen data.

During final evaluation, I conducted a test of the model using data augmentations provided by the CIFAR-10-C dataset. This dataset is comprised of 15 transformations at five severity levels and is designed to test the robustness of the model. For a full picture of the project's history, I compared final model with previous versions to determine if any generalization was lost during development. The results can be found in Figures 6-8.

**Figure 6: CIFAR-10-C Heatmap**

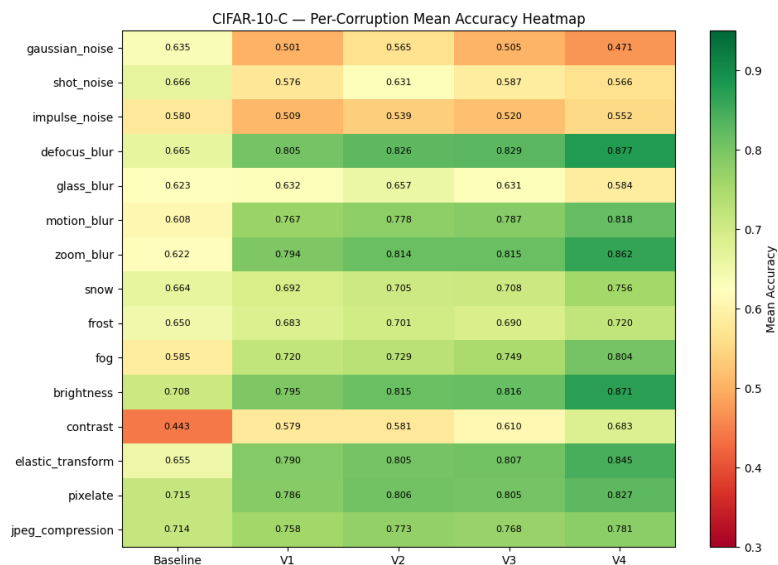


Figure 7: CIFAR-10-C Accuracy Comparison

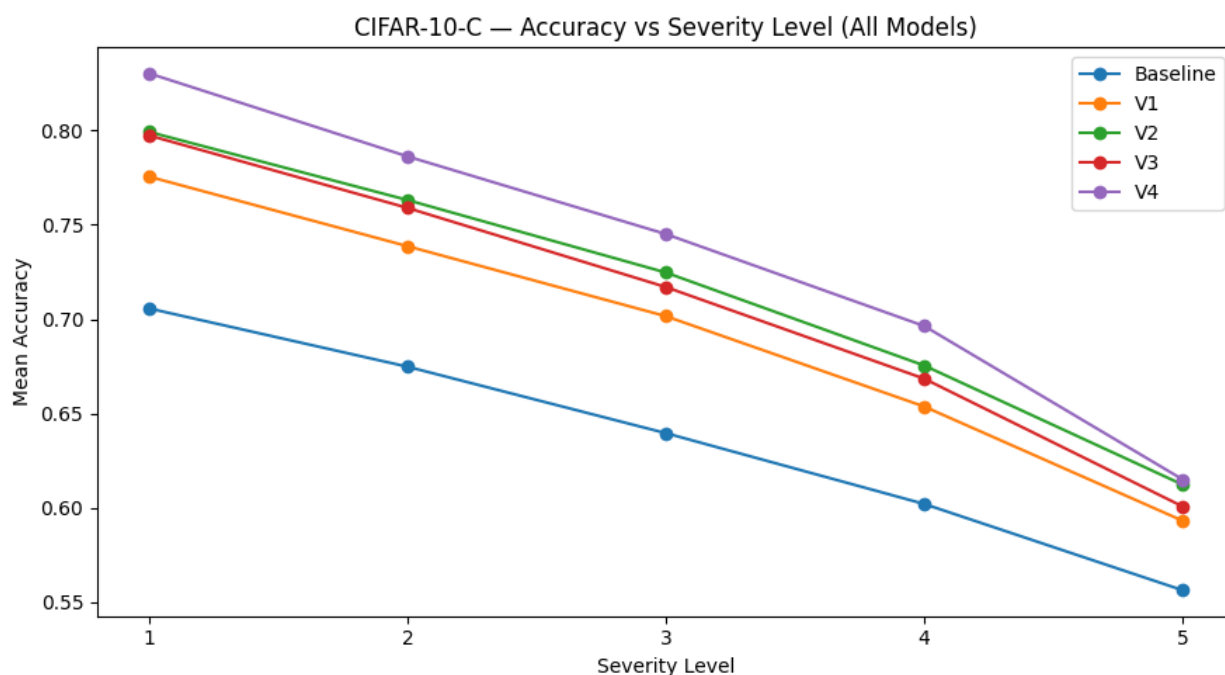
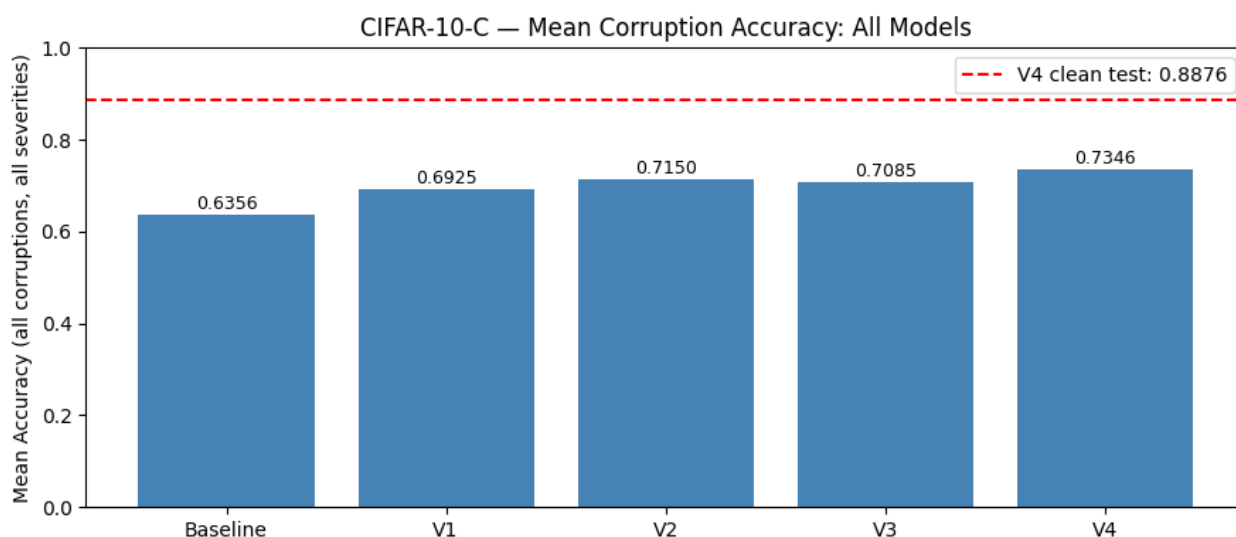


Figure 8: CIFAR-10-C Mean Corruption Accuracy Comparison



The results showed significant improvement in Version 4 up until severity Level 5; a decrease in accuracy for Gaussian noise, shot noise, and glass blur; and an improved, but still relatively low, impulse noise accuracy. These results are described in more detail in Part J.

### G. Describe the techniques used to mitigate imbalanced datasets and their effectiveness.

As the CIFAR-10 dataset contains an equal number of training and test samples per class, imbalance mitigation was unnecessary for this project. However, if the model was being trained on an

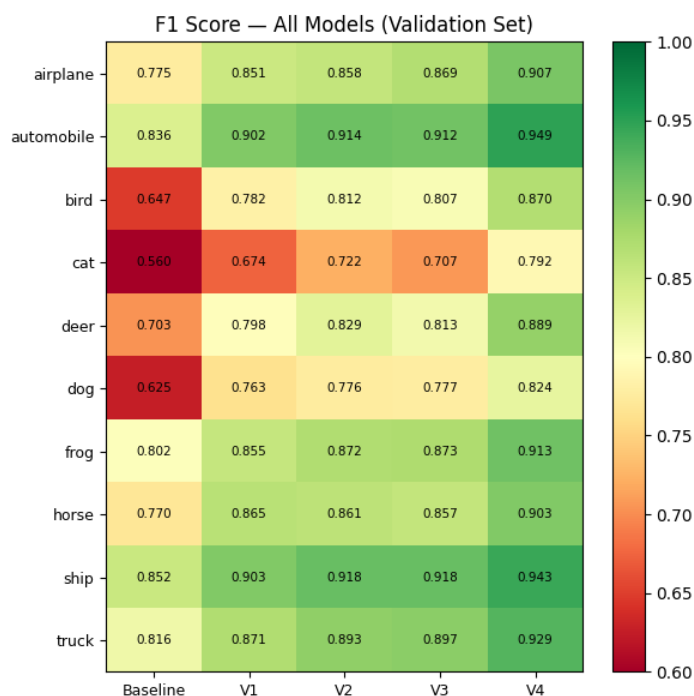
imbalanced dataset, the best course of action would be to collect more data related to the underrepresented class. This can often require significant time and effort but is by far the most effective way to address the issue. In the absence of additional data, one can oversample a minority class by using data augmentation to increase the number of different images for the underrepresented class (for example, rotating or cropping the images to create new inputs). This is effective in increasing the minority class while keeping the data diverse (Geeks for Geeks, 2026). Another option is to undersample a majority class, which may be helpful if there is only one or two classes with a large amount of data (Geeks for Geeks, 2026). However, it is effectively throwing away usable resources which is ultimately inefficient.

Finally, one of the most common techniques is to apply class weights, which can be used to increase penalties for misclassifying underrepresented groups (Google, 2025a). This allows the use of all available data regardless of balance but can destabilize training if the weights get too high.

## H. Explain how error analysis was used to identify common misclassification types and potential reasons for those errors.

To conduct error analysis, I used the confusion matrix on both validation and test sets of the final model, in tandem with per-class F1 score, precision, and recall reports, to identify error-prone classes. All previous versions were also tracked for accuracy, F1 score, precision, and recall to determine if they were performing better or worse in specific categories. Finally, I used the CIFAR-10-C to conduct a robustness analysis on the final model.

Cat was the most problematic class, primarily due to similarities with dogs, which performed the second worst. Cats misclassified as dogs and vice versa made up the bulk of the misclassifications, with other animal classes showing substantially more misclassifications than vehicles classes. This implies that the presence of shared characteristics such as ears, fur, legs, and body types makes animals more difficult for the model to distinguish between. However, animals were still within reasonable F1 scores, as shown in the heatmap below, with the bulk of the issue lying in cat and dog classification. Further details on these results can be found in Part I.2.



The results of the CIFAR-10-C analysis show that the final model continues to struggle with Gaussian noise, shot noise, impulse noise, and glass blur. This implies that the model relies heavily on edge information and visual detail when making classifications. While the final mean accuracy for CIFAR-10-C shows substantial improvement over previous models, the corruption types in question show that additional improvements are possible.

**I. Prepare a final report that describes the functionality of your neural network. Do the following in the report:**

**1. Explain the role and functionality of *each* major component of your neural network.**

The network is comprised of nine key components:

1. An input layer for receiving images from the dataset based on their universal dimensions (32 x 32 x 3)

This layer is used to receive input from the dataset, with the shape designated to fit the size of the images, which in this case is universal (32 x 32 pixels, three color channels).

2. Data augmentation layers to increase the variation of the training data.

There are four layers in this component that each perform a different transformation: RandomFlip which randomly flips images horizontally; RandomRotation, which randomly rotates images  $\pm 36^\circ$ ; RandomZoom, which randomly zooms in or out of the image by 10%; and RandomContrast, which adjusts contrast by  $\pm 10\%$ . These transformations improve model generalization by introducing additional variation to the training data, effectively increasing the diversity of the dataset, reducing memorization, and preventing overfitting.

3. Eight convolutional layers to analyze patches of the images to extract increasingly complex features from the input images.

The convolutional layers of the final model use a VGG-inspired architecture consisting of four blocks of two 3x3 layers. Each convolutional layer in the model uses a 3x3 kernel size for each filter. The filters in the first block are the base parameter determined by Bayesian optimization. Subsequent blocks increase the number of filters by a factor of two, four, and eight. This is done to increase the number of feature maps, enhance feature depth, and incorporate more advanced feature learning as the input travels through the model (Simonyan & Zisserman, 2015). The activation function for each convolutional layer is ReLU, with padding equal to 'same' to preserve the input's shape in the output layer. The role of these layers is to identify features while preserving spatial dimensions, allowing the model to learn patterns more efficiently than traditional feed-forward networks (Maurício, Domingues, & Bernardino, 2023).

4. A batch normalization layer following each convolution layer to stabilize training

The batch normalization layer calls the BatchNormalization function, which normalizes each convolutional layer's output, standardizing the activations produced by previous layers and learning scaling parameters during training. (Geeks for Geeks, 2025c). This differs from fixed normalization by adapting the range to what is producing the best learning results. The role of these layers is to stabilize training by maintaining consistent activation distributions, thereby improving convergence speed and providing a mild regularization effect (Geeks for Geeks, 2025c).

5. Several pooling layers to reduce spatial dimensions and computational complexity.

A single max pooling layer follows each pair of convolutional layers apart from the last pair which is followed by a global average pooling layer (see below). Each layer uses a 2 x 2 sliding window and determines the maximum value for each region, effectively reducing the spatial dimensions of the input by 50%. In addition to reducing parameters, pooling retains the strongest activations within each region. In this model's architecture, the pooling layers eventually bring the dimensions of the images down to 4x4. The role of the pooling layers is to reduce spatial dimensions and parameter sizes, making the model more efficient while preserving the most important information for making predictions (Nanos, 2025).

6. A global average pooling (GAP) layer to reduce parameter size.

Used as an alternative to flatten, GAP reduces parameter sizes by averaging each feature map to a single value (i.e., a 4x4 grid of values would be averaged to produce a single value). This is preferred for this model over flattening due to its significantly reduced parameters, reduced overfitting risk by nature of fewer weights, and the fact that GAP is less sensitive to the spatial location of features (Nanos, 2025). The role of this layer is to reduce the dimensionality of feature maps before classification, only preserving the most important information to improve performance.

7. A dense neural network layer to map the results of the image processing to the labels.

This layer serves as the final hidden layer before classification. As a dense layer, each input neuron is connected to each output neuron. With ReLU activation, each neuron only retains positive values (Gupta, 2021). The layer takes the extracted features from the convolutional layers and combines them into global patterns to determine the most accurate classification (Zhao, et al., 2024). The hyperparameters were determined using Bayesian optimization.

The layer uses 512 features received from global average pooling and outputs 256 neurons with 256 biases, leading to 131,328 total parameters. This dimensionality reduction leads to far less parameters than the amount created by using flatten, which is equal to 2,097,408. This parameter reduction helps decrease computational demand and overfitting risk.

8. A dropout layer to prevent overreliance on individual neurons and prevent overfitting

This layer randomly sets 30% of neurons to zero during training to ensure the model does not depend too heavily on specific neurons. This ensures that each neuron can develop robust features independently (Geeks for Geeks, 2025d). Dropout is then removed entirely for validation and testing. Bayesian optimization determined that 30% was the ideal hyperparameter, providing the best balance between overfitting and underfitting.

9. An output layer to convert learned features to classification outputs.

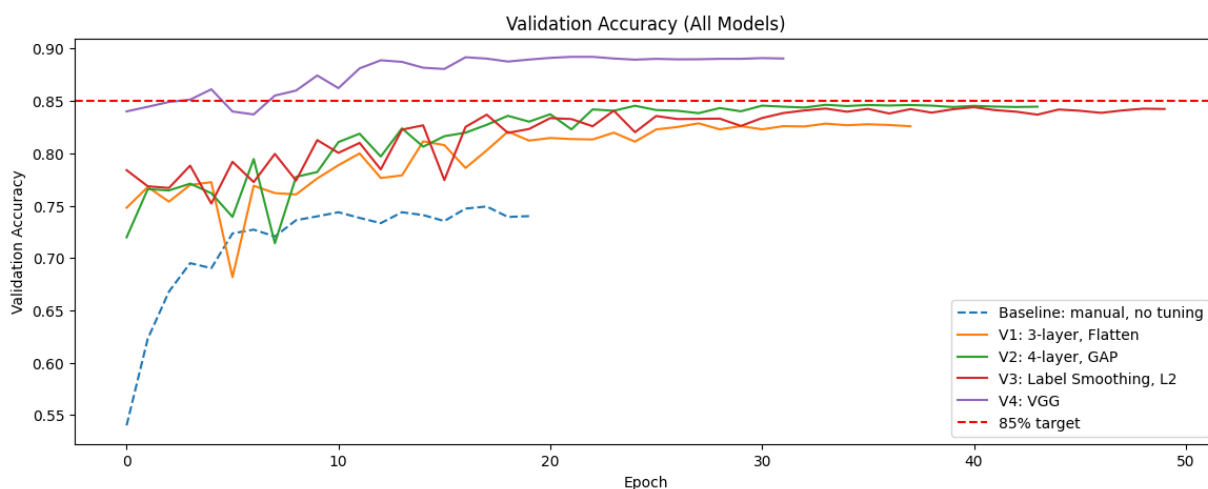
This layer converts the 256 features of the dense layer into probabilities for the classes in the CIFAR-10 dataset. It computes the raw scores and uses SoftMax to exponentiate each value and normalize them, so the sum of the probabilities is one (Geeks for Geeks, 2025e). The class with the highest SoftMax probability is selected as the final prediction.

## 2. Discuss the performance metrics used to evaluate the model and provide a detailed analysis of the results.

The performance metrics used to evaluate the model were accuracy and F1 score, precision, and recall per class. Precision and recall are compared to the confusion matrix for a better understanding of the results.

Accuracy was chosen as the primary metric because CIFAR-10 contains an equal number of images per class. As there is no class imbalance present, accuracy is a reliable measure of the overall performance of the model.

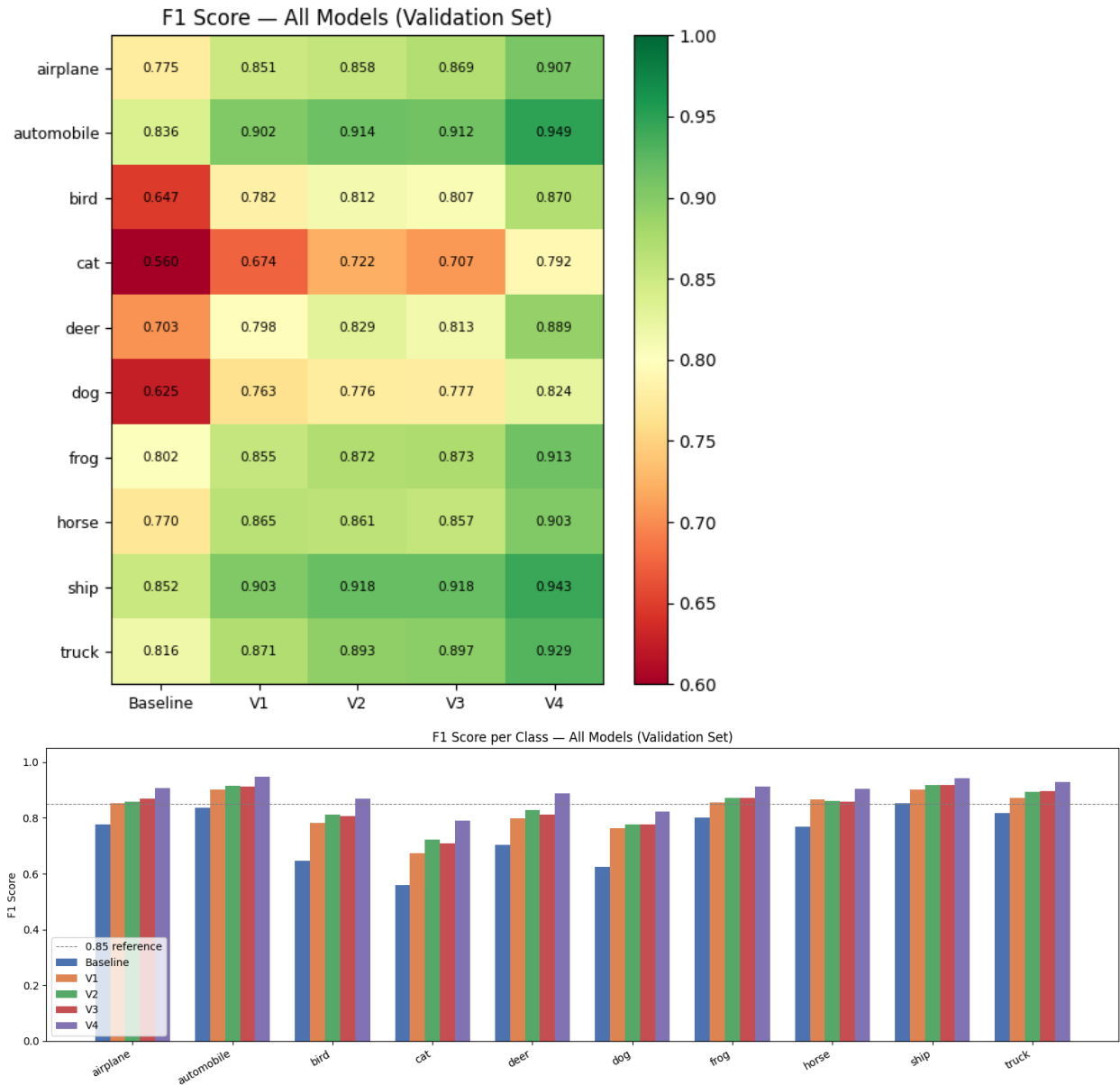
**Figure 9: Validation Accuracy Comparison**



F1 score was used to ensure that there were no outlying misclassification issues or troublesome classes. F1 score takes the harmonic mean of precision and recall to provide a measurement of classification performance (Scikit-learn, 2026). The use of F1 score is necessary to determine how well the model performs between classes, which accuracy alone does not provide. For example, a model trained on CIFAR-10 may have an accuracy of 90%, but the bulk of the 10% of errors might be in a single class, meaning there is an issue with the model that needs to be addressed. If comparing it to a model with 88% accuracy, we may choose the 90% model even if the 88% model proves to generalize better between classes.

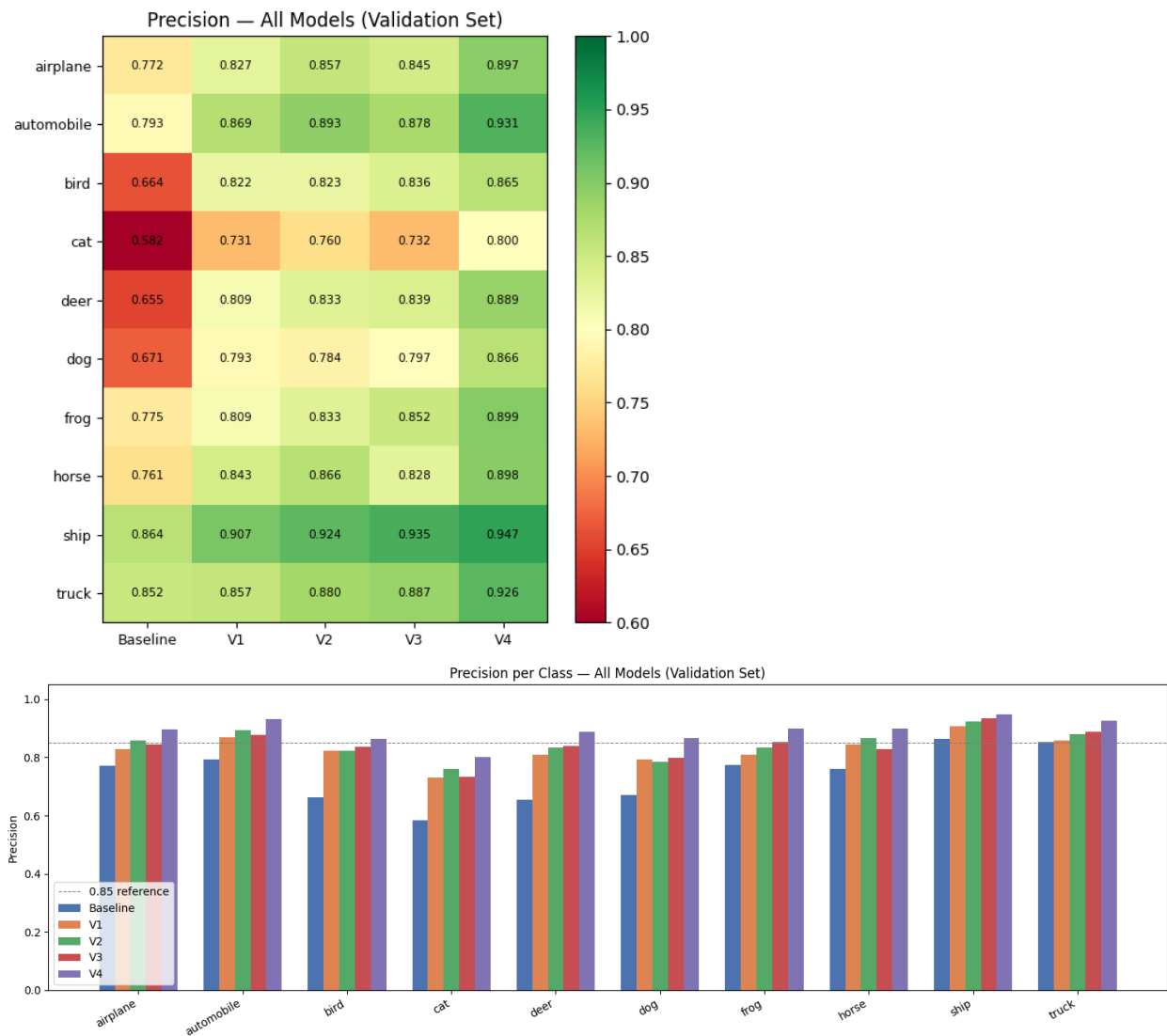
The results for all models showed a difficulty with correctly classifying cats and dogs, likely because of their similarity in appearance. Both classes fell below 85% in validation. The highest F1 score achieved was with automobiles and ships, likely because they have more visually distinctive features. The rest of the classes performed relatively similarly and above 85% for all model versions. The baseline model performed relatively poorly outside of automobiles and ships. The large increase in performance between Version 4 and the preceding models shows that the VGG-inspired architecture significantly outperforms previous versions, particularly in the more difficult classes.

Figure 10: F1 Score Comparison (Heatmap and Bar Chart)



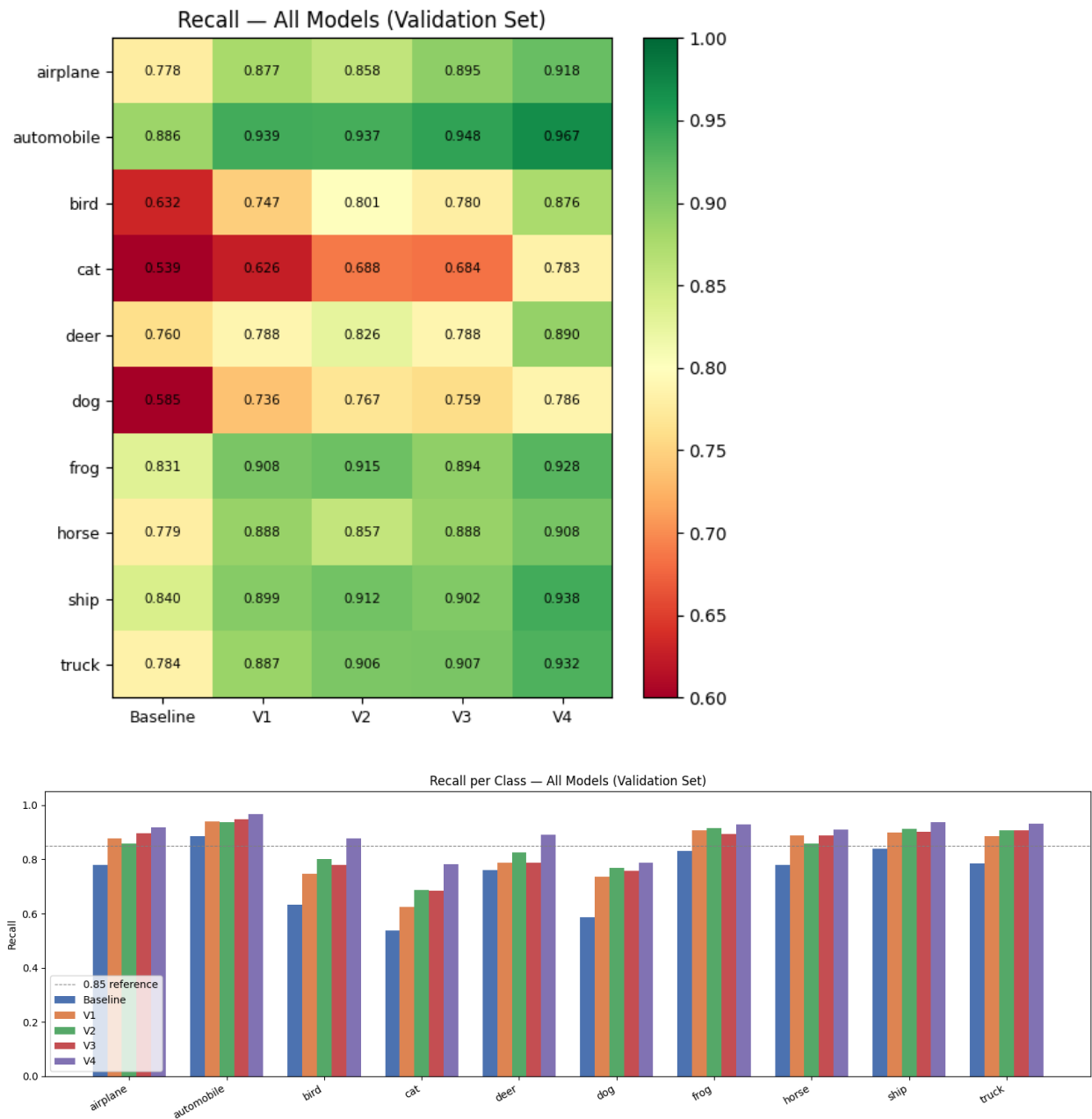
Precision measures the proportion of positive predictions that are correct. In tandem with recall, this metric is used to determine how conservative the models are in determining the results (Google, 2026b). For example, a high precision would mean the model accurately predicts dogs more than misclassifying, while a low score would infer that the model is over-predicting a class.

Figure 11: Precision Comparison (Heatmap and Bar Chart)

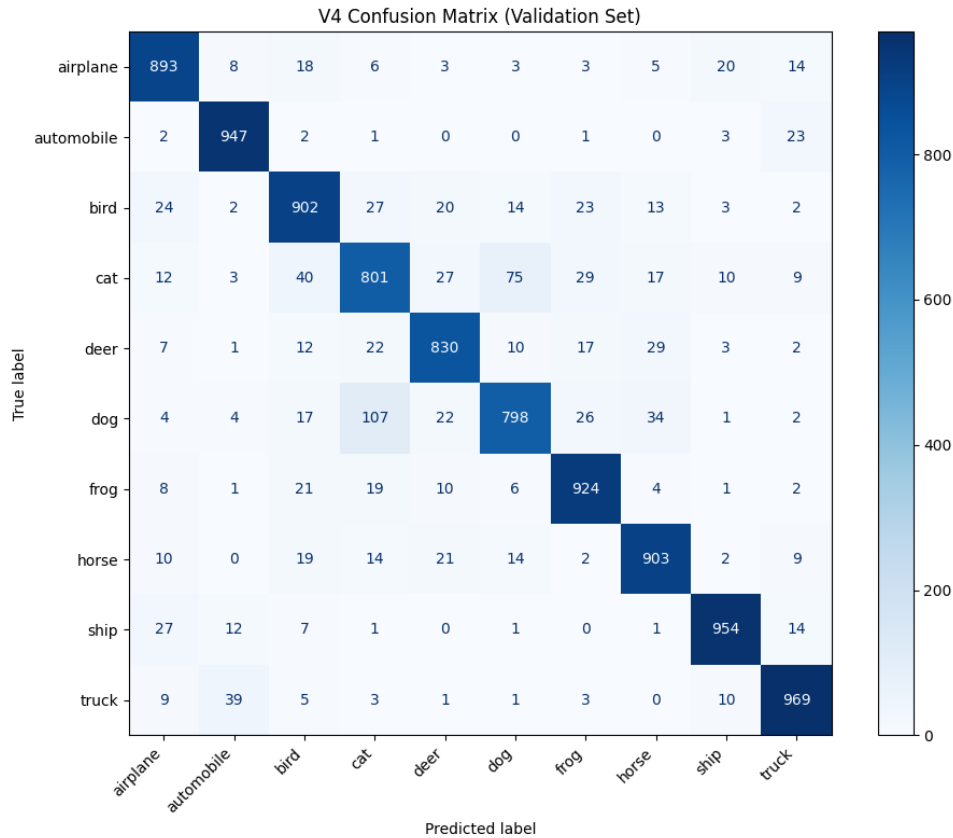


Recall measures the proportion of true classes that are correctly predicted. For example, a low recall would imply that the model has difficulty recognizing a class, while a high recall implies that false negatives are minimal (Google, 2026b). Precision and Recall are both needed to evaluate the model, as they explain why the F1 score is at its current value.

Figure 12: Recall Comparison (Heatmap and Bar Chart)



Precision and recall were then analyzed alongside the confusion matrix to determine the classes predominantly responsible for the classification errors.

**Figure 13: Confusion Matrix for Version 4 (Validation set)**

The results of this analysis show a few issues in particular:

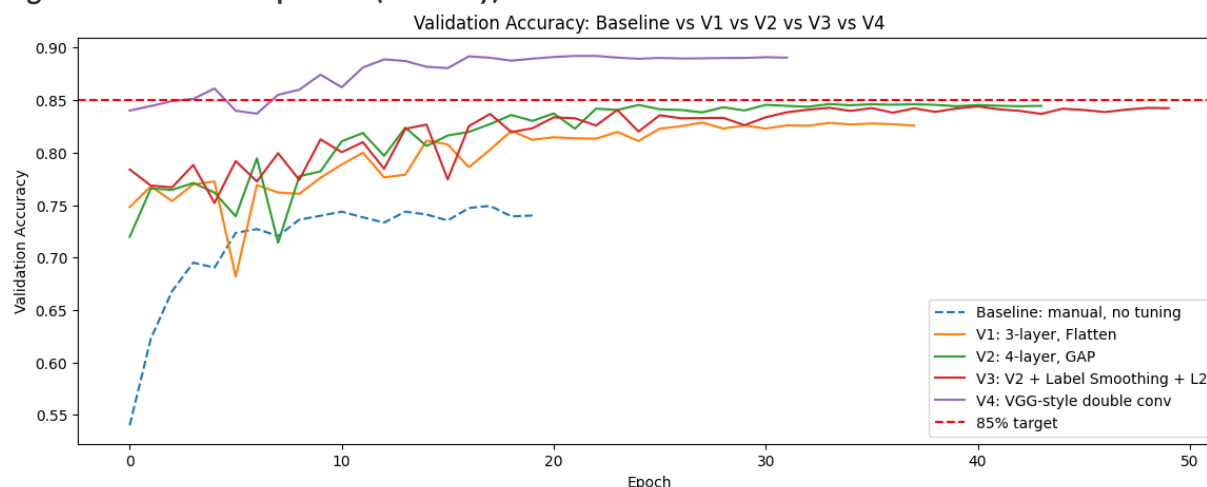
1. The model has the most trouble with the cat class, showing low precision and recall in comparison with other classes. Dogs make up the bulk of errors, with 75 misclassifications. This is followed by birds at 40, frogs at 29, deer at 27, and results in the teens for all other classes besides automobiles (the strongest class) and trucks.
2. The dog class shows relatively high precision but low recall, implying that the model is more conservative when selecting dogs, but misidentifies several of them. Looking at the confusion matrix, we see that dogs are misclassified as cats 107 times, which is the most significant misclassification instance in the heatmap. There are also 34 cases of misidentifying dogs as horses, 22 misclassifications of both deer and frogs, and 21 misclassifications as birds. This shows the model has difficulty differentiating between dogs and cats, while having some difficulty differentiating between dogs and other animals in the dataset.
3. The frog class shows a higher recall than precision, showing that the model tends to over-predict frogs. This is evident in the confusion matrix, where dogs, cats, and birds are misclassified as frogs over 20 times and horses are misclassified 17 times.

4. The vehicles in the dataset (airplanes, automobiles, ships, and trucks) show very balanced precision and recall, meaning they are the most consistent and least error-prone of the classes.

Overall, the analysis of the model shows a significant increase in accuracy, F1 score, precision, and recall for Version 4, showing that the VGG-inspired model is the strongest candidate. While there are still issues with animal classes, particularly cats and dogs, Version 4 still shows strong results across most classes and a much stronger performance in trouble classes. This indicates that the model has generalized well given its constraints and meets the project's objectives.

3. Compare the performance of your final model with other baseline models or previous models on the same dataset.

**Figure 14: Model comparison (accuracy)**



Overall, the performance of the model was significantly increased with the introduction of VGG-inspired architecture, which uses two smaller convolutional layers prior to pooling rather than one large convolutional layer. The final version has a validation accuracy of 89.2%, test accuracy of 88.7%, and acceptable F1 scores for all classes, with only cats and dogs falling short of the 85% threshold. The final model achieved the strongest overall performance across most metrics, with the only downside being the significant training time involved (over 16 hours).

Compared to earlier versions, the baseline model resulted in approximately 74.9% validation accuracy. The F1 score for all classes except ships fell below the 85% threshold, with precision and recall primarily low with animal classes. This indicated that the model had limited feature extraction and poor generalization.

Version 1 introduced early stopping with a patience of 10, three transformation layers, a batch normalization layer, and Bayesian optimization for hyperparameters including filters, dropout, and learning rate. This version improved accuracy to approximately 82.6%, which was a significant improvement over the baseline. The model also showed improvement in F1 scores for all classes but still failed to meet the 85% threshold for birds, cats, deer, and dogs.

Version 2 added an additional convolutional layer, replaced flatten with GAP, and added an additional transformation layer for random contrast. This version increased accuracy to approximately 84.6%. However, F1 scores showed little improvement, and still fell below the 85% threshold for birds, cats, deer, and dogs.

Version 3 added L2 regularization and label smoothing to the model. This resulted in approximately 84.4% accuracy with mixed changes in precision and recall across classes. This means that the model was already suitably regularized and additional measures may have been unnecessary.

Overall, the final model produced the most consistent improvements across metrics. While weaknesses remain in distinguishing animal classes, particularly cats and dogs, it provides the most balanced performance across all categories. The tradeoff for this performance was increased computational cost, with significantly longer training times compared to past versions.

#### 4. Compare the hyperparameter and activation functions of the final model with those of baseline models on the same dataset.

The hyperparameters of the final model changed significantly from previous iterations, primarily due to the use of Bayesian optimization and a change in architecture. The following is a breakdown of the final model's code by section, accompanied by comparisons to previous models.

##### a. input and transformation layers

```
model = Sequential()
    model.add(Input(shape=(32, 32, 3)))
    model.add(RandomFlip('horizontal'))
    model.add(RandomRotation(0.1))
    model.add(RandomZoom(0.1))
    model.add(RandomContrast(0.1))
```

The input shape remained the same throughout iterations, but transformation layers were not included until Version 1, with Random flip, rotation, and zoom added. Random contrast was included in Version 2 to further help with generalization. The hyperparameters of these layers were not adjusted during development, as I felt the small adjustments were suitable and higher values could lead to unnecessary complexity.

##### b. Convolutional layers

```
base = hp.Int('base_filters', min_value=32, max_value=64, step=32)

# Block 1: 32x32 → 16x16 (double conv)
model.add(Conv2D(base, (3, 3), activation='relu', padding='same'))
model.add(BatchNormalization())
model.add(Conv2D(base, (3, 3), activation='relu', padding='same'))
model.add(BatchNormalization())
model.add(MaxPooling2D((2, 2)))

# Block 2: 16x16 → 8x8 (double conv)
model.add(Conv2D(base * 2, (3, 3), activation='relu', padding='same'))
model.add(BatchNormalization())
model.add(Conv2D(base * 2, (3, 3), activation='relu', padding='same'))
model.add(BatchNormalization())
model.add(MaxPooling2D((2, 2)))
```

```

# Block 3: 8x8 → 4x4 (double conv)
model.add(Conv2D(base * 4, (3, 3), activation='relu', padding='same'))
model.add(BatchNormalization())
model.add(Conv2D(base * 4, (3, 3), activation='relu', padding='same'))
model.add(BatchNormalization())
model.add(MaxPooling2D((2, 2)))

# Block 4: 4x4, no pool – GAP collapses spatial dims (double conv)
model.add(Conv2D(base * 8, (3, 3), activation='relu', padding='same'))
model.add(BatchNormalization())
model.add(Conv2D(base * 8, (3, 3), activation='relu', padding='same'))
model.add(BatchNormalization())
model.add(GlobalAveragePooling2D())

```

The baseline model used manually selected hyperparameter values that are commonly used for CIFAR-10 classification. The model had three convolutional layers with 64 filters, a 3x3 kernel size, ReLU activation, and padding set to same. Each layer was followed by a max pooling layer with 2x2 kernel size, apart from the last layer which was followed by a flatten layer. The kernel sizes, activation, and padding remain the same throughout iterations.

In Version 1, I used Bayesian optimization to determine the best filters for each convolutional layer, with search ranges of 32-64, 64-128, and 128-256 respectively with 32, 32, and 64 incremental steps respectively. I also added batch normalization after each layer.

In Version 2, I added a fourth convolutional layer with Bayesian optimization determining a filter count between 256-512 with 128 steps. In addition, flatten was replaced with global average pooling to reduce the parameter count significantly, improving computational performance.

No changes were made to this section in Version 3, but Version 4 made a significant change to the model as shown above. Instead of four convolutional layers, I decided to divide them into 8 smaller layers to improve performance to create a VGG-inspired architecture. As explained in Part I.1, the filter in the first pair of layers is determined by Bayesian optimization which is multiplied by two, four, and eight for each following pair respectively.

I chose to keep the activation functions the same throughout the process. For the convolutional and hidden layers, I used ReLU. ReLU is a common activation function for image classification models due to its simple function of  $f(x) = \max(0, x)$ , which states that a negative output would return 0 and a positive output would be equal to the input (Krishnamurthy, 2024). This leads to sparse activations, which can improve feature selection and computational efficiency.

The reason for using ReLU was two-fold: first, the computational demand of ReLU is significantly less than other options such as tanh, particularly with a model that has several convolutional layers (Krishnamurthy, 2024). The importance of keeping ReLU to mitigate computational demand became more valuable as new versions were created. Secondly, with the use of batch normalization, ReLU helps reduce vanishing gradient issues because positive inputs retain a constant gradient of 1, allowing data to move more effectively through deep networks (Krishnamurthy, 2024).

### c. Hidden, dropout, and output layers

```
model.add(Dense(hp.Int('dense_units', min_value=256, max_value=512, step=128),
activation='relu'))
    model.add(Dropout(hp.Float('dropout', min_value=0.2, max_value=0.5, step=0.1)))
    model.add(Dense(10, activation='softmax'))
```

The corresponding section of the baseline consisted of one dense hidden layer with 128 neurons and ReLU activation, a dropout layer with a rate of 30%, and an output layer with SoftMax activation. The activations for both the hidden and output layers remained unchanged throughout iterations, and 30% remained the dropout rate as Bayesian optimization consistently selected it as the best value.

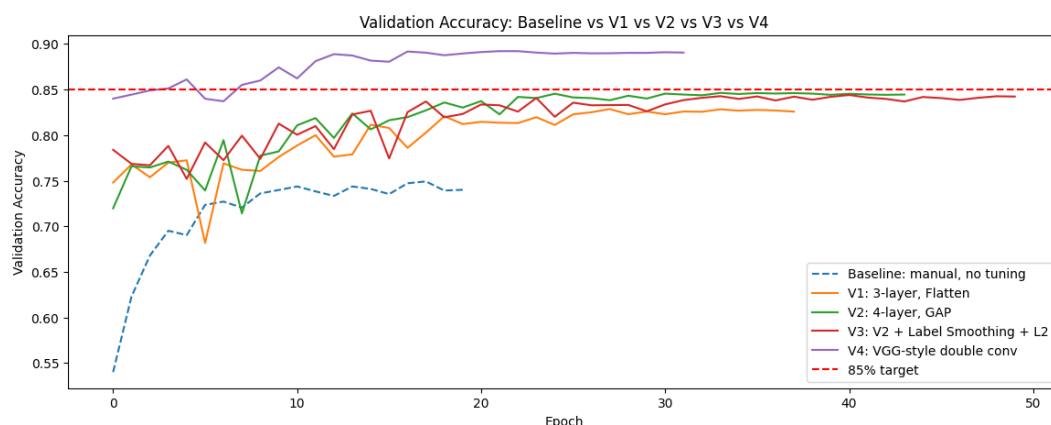
As stated under Part I.4.b, ReLU was used as the activation function for the hidden layer. The SoftMax activation function was used for the output layer of the baseline and each subsequent version. SoftMax was preferred for this project because it addresses a multiclass classification problem with a single label per image. SoftMax converts numerical outputs for each class to numbers that fall between 0 and 1, which equal one when added together. This means the numbers serve as a probability of the best classification and enforce competition between classes (Geeks for Geeks, 2025e). Additionally, SoftMax naturally works well with categorical cross-entropy, which is standard loss function for multiclass classification problems (Geeks for Geeks, 2025e).

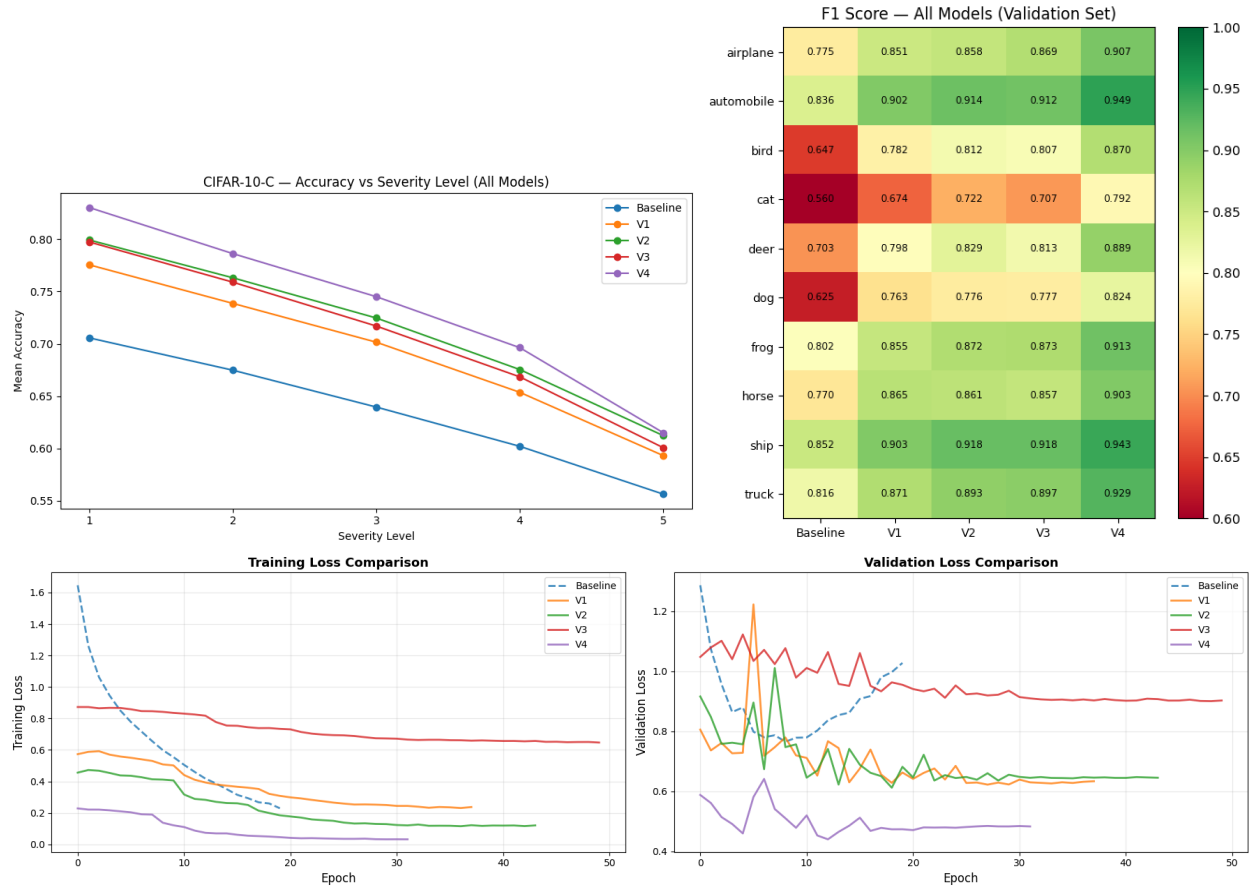
Version 1 introduced Bayesian optimization to determine the ideal unit count and dropout rate. The range for the unit count was initially 128-512 with 64 steps. The search range was reduced to 256-512 with 128 steps in Version 4 to better match the feature representation of the VGG-inspired architecture.

No other changes were made to this section prior to the final model.

##### 5. Explain the results found, including accuracy for different epochs, and visual representations for observations made for performance.

**Figure 15: Visual representations of performance (Accuracy, F1 Score, Training and Validation Loss, and CIFAR-10-C Accuracy)**





The graphs and heatmap shown under Figure 15 show the training results for each iteration of the model. These show the accuracy and loss curves, F1 score, and CIFAR-10-C results for each version, providing insight into the impact of each architectural and hyperparameter change.

The baseline model was assigned manually selected parameters typically used for computer vision models, with each convolutional layer provided the same number of filters. For the baseline, 20 epochs were run without early stopping measures. This resulted in approximately 74.9% validation accuracy.

Version 1 introduced early stopping with a patience of 10, three transformation layers, a batch normalization layer, and Bayesian optimization for hyperparameters including filters, dropout, and learning rate. For Version 1, 50 epochs were run, resulting in the model achieving approximately 82.6% accuracy at epoch 28. For all subsequent versions, a maximum of 50 epochs with a patience of 10 was used for training.

Version 2 added an additional convolutional layer, replaced flatten with GAP, and added an additional transformation layer for random contrast. Version 2 returned its best results at epoch 34 with approximately 84.6% accuracy.

Version 3 added L2 regularization and label smoothing, with the best results being approximately 84.4% accuracy at epoch 41, a slight decrease.

Version 4 replaced the convolutional layers with a VGG-inspired architecture. The maximum training epochs remained unchanged and the best results were achieved at epoch 22 with approximately 89.2% accuracy.

Overall, the results showed consistent improvement apart from Version 3, where L2 regularization and label smoothing decreased accuracy, suggesting that Version 2 was already sufficiently regularized. The largest gains were from Bayesian optimization and early stopping in Version 1, followed by VGG-inspired architecture introduced in Version 4. Version 2's use of GAP and an additional convolutional layer produced modest gains in accuracy while reducing overfitting risk and parameter count. Additionally, F1 scores between the baseline and Version 4 increased significantly, showing the model consistently improved its ability to generalize, apart from the slight regression in Version 3. Validation and training loss were also lower in Version 4 than previous models, showing the effectiveness of VGG-inspired architecture combined with the regularization applied throughout development. Finally, the CIFAR-10-C dataset was used to compare the robustness of the final model to previous versions, showing an improved ability to generalize to corrupted inputs.

**6. Discuss any ethical considerations related to the dataset or the application of the model, such as privacy concerns or potential biases.**

In terms of the dataset, CIFAR-10 consists entirely of 32x32 images. This low resolution does not accurately reflect the quality and complexity of images captured in real-world environments. As such, the model is unlikely to maintain the same level of performance when deployed unless it is further trained on more complex inputs and industry-specific data. Developers should clearly communicate these limitations when recommending the model for real-world applications to ensure stakeholders understand the model's current scope and ability.

Additionally, CIFAR-10 is a curated subset of the Tiny Images dataset, which contains 80 million 32x32 images (Prabhu & Birhane, 2021). In 2020, the Tiny Images dataset was withdrawn by the Massachusetts Institute of Technology for containing “derogatory terms as categories and offensive images” (Torralba, Fergus, & Freeman, 2020). Additionally, the dataset was called into question for privacy concerns, with many images of people and places being included without consent (Prabhu & Birhane, 2021). While the CIFAR-10 is a curated selection of the greater dataset, the fact that it is derived from a source with questioned ethical practices highlights the importance of responsible data collection and curation.

In terms of the model itself, the technical limitations discovered via F1 score and testing on CIFAR-10-C pose ethical concerns in real-world applications. For example, while the model can produce high F1 scores for vehicle classes, animal classes still possess generalization issues, particularly for dogs and cats. With this in mind, the model could likely pose ethical concerns for animal classification in real-world scenarios requiring impactful decision-making (e.g., conservation efforts, which rely on accurate information to make informed decisions). Because of this, further training and fine-tuning may be required before the model could be ethically used for animal classification in general. This issue also highlights how models can develop biases towards certain classes based on characteristics of the training data, causing uneven performance across categories.

Additionally, CIFAR-10-C testing results show the model is vulnerable to Gaussian noise, shot noise, impulse noise, and glass blur. These weaknesses must be communicated to users to ensure they do not become overconfident in the model's predictions. In this respect, developers should validate performance in real-world conditions and share their findings prior to deployment.

**7. Describe how the training model could be deployed in a real-world scenario, including considerations for scalability and integration with other systems.**

This model can be deployed in a real-world scenario provided that it is further trained on industry-specific knowledge. For example, this model can provide the foundation for a vehicle classification system. In this scenario, the model could be retrained on a specialized automotive dataset to detect the make, model, year, and color of vehicles photographed at an intersection for the purpose of traffic monitoring or law enforcement. However, this would require far more domain-specific knowledge than CIFAR-10 provides. As the model has shown strong performance on automobiles and trucks, it provides a promising foundation for this application.

This model also has potential for classifying animals; however, precision and recall issues remain, particularly with dogs and cats. The model demonstrates promising results in animal classification but would still require additional training data and fine-tuning, such as class weighting, before being suitable for real-world deployment. It would also be particularly difficult to justify the use of the model in its current form for making impactful decisions, such as conservation efforts, without significant improvement to the robustness of the model, as explained in Part I.6.

In terms of scalability, the model would likely need retraining on higher-resolution images and a larger dataset. Considering that the model's training time was over 16 hours, retraining on this data would further increase computational requirements and training time. However, the development process of adapting the model can be streamlined with the use of Bayesian optimization, provided the ranges and steps are adjusted appropriately.

Regarding integration with other systems, the model was built in Python 3.11.15 with the Keras SDK, which supports integration with a wide range of systems and deployment environments. The model's code can be further containerized and utilized on web applications and/or edge devices using a cloud computing service such as AWS or Azure, enabling deployment across a variety of applications and devices.

**8. Suggest possible improvements that could be made based on the performance comparison, such as hyperparameter tuning, additional data preprocessing, or use of more advanced architectures. Mention if any errors occurred and how they were resolved.**

There are several improvements that could be made to the model based on the performance comparison.

In terms of hyperparameters, additional fine-tuning could be explored by increasing the search space and/or trials for Bayesian optimization. The final model was conservative with ranges and trials to avoid computational expense, but additional resources could be devoted to exploring a wider range of options.

The addition of extra transformation layers to train for noise and blur could lead to an improvement in CIFAR-10-C scores. The final model only includes flip, zoom, rotation, and contrast augmentations, which is likely why CIFAR -10-C mean accuracy for noise and glass blur were significantly less than other corruptions.

For training, the use of SGD + Momentum as an optimizer in place of Adam could lead to a performance improvement over time. While Adam produced strong results and converged efficiently, SGD + Momentum could possibly achieve higher accuracy with sufficient training time. This is because SGD + Momentum converges more slowly, leading to solutions that may generalize better to

unseen data (Lee, 2024). However, this optimizer would require additional epochs, fine-tuning, and training time for convergence. As the final model is already performing at 88.7% accuracy, the improvements would likely be modest. However, the optimizer remains an option for future experimentation.

In terms of architecture, a logical next step would be to explore residual networks (ResNets). This architecture utilizes skip connections, which allow data to bypass intermediate layers, enabling the model to focus on learning refinements to features rather than new representations (Zhao, et al., 2024). These skip connections allow deeper networks to be trained without suffering from degradation problems (Zhao, et al., 2024). This architecture may provide additional gains because the final model showed a correlation between increased convolutional layers and improved performance. Additionally, using transfer learning with pretrained ResNet models could improve the accuracy substantially, but this would require a different approach than the custom architecture of the model.

A vision transformer (ViT) could also be considered, as the architecture has been shown capable of sometimes exceeding CNN accuracy in computer vision tasks (Maurício, Domingues, & Bernardino, 2023). ViTs divide images into patches and use the attention mechanism to determine the most important regions of the image for classification, allowing the model to perceive long-range relationships across an image (Maurício, Domingues, & Bernardino, 2023). However, the architecture would require significantly more data and computational resources, making it less practical for the CIFAR-10 dataset without transfer learning or significant changes in architecture.

Errors include previous tunings being overwritten at a point when several versions were being tested in tandem. The issue was resolved by disabling the `overwrite = true` setting in the tuner configuration. Another issue was the lack of checkpointing and early stopping procedures, which I did not include in the baseline and my first run of Version 1. After identifying the issue, Version 1 was retrained with these procedures enabled after clearing the training session. These procedures were retained through all subsequent iterations.

## **J. Recommend a course of action based on your results and discuss *any* challenges you faced during the project such as technical difficulties, data issues, or unexpected results.**

Based on the results of this model, I would recommend further improving the model's robustness before considering deployment in real-world applications. The CIFAR-10-C testing showed significant weaknesses with noise and blur corruptions. Future work should include applying additional augmentation techniques and hyperparameter tuning. I would like to see the mean corruption accuracy exceed 75% across all corruption categories. Once this threshold is met, the model should be evaluated on additional datasets to further validate its ability to generalize.

Technical difficulties included the time it took to complete training, which, beginning at Version 2, required overnight training runs. Thankfully there were no interruptions in training, but checkpointing procedures were in place to ensure the model could continue where it left off.

The dataset was straightforward, as CIFAR-10 is balanced between classes, each image is equally sized, and the test-train split is provided by default with pre-randomized and categorized data. However, determining the level of noise, occlusion, etc. proved difficult due to the 32x32 size of the images. In all cases, the issues flagged in my data cleaning were the result of poor resolution, not corrupted images. This required me to view each flagged image and determine that the abnormalities were false positives.

An unexpected result in training was encountered when applying L2 regularization and label smoothing to the model. This resulted in a decrease in accuracy and per class metrics in the model, which suggested that the model was already sufficiently regularized and additional regularization was constraining learning rather than improving generalization. It was at this point I had to come up with alternatives in the architecture to meet my 85% target, which led me to VGG-inspired architecture. While I expected it to perform better, I was surprised at how much of an improvement the final model was over previous iterations. The model's accuracy increased by approximately 5%, with nearly all other metrics being significantly better than any other iteration.

Finally, I was surprised by the results of the CIFAR-10-C model comparisons. Version 4 showed improvement over other models in all categories except gaussian noise, shot noise, and glass blur, which each decreased in accuracy from 2-5 percentage points from previous iterations. While I am uncertain of the cause, several explanations exist:

1. I did not include transformation layers in the model for noise, which could have helped the model determine predictions on noise-transformed images more accurately.
2. While Version 4's additional convolutional layers allow the model to learn richer features, deeper architecture can also amplify the effects of distortions by repeatedly transforming corrupted inputs. The fewer layers of previous models could therefore be making them more noise tolerant while still being outperformed in other metrics.
3. The final model's improved performance may be indicative of the model being more specialized at extracting features from clean images. While this improved accuracy overall, it may have increased sensitivity to corruptions that obscure those features.

#### **K. Discuss the limitations of the current model, such as its generalizability, computational requirements, or dependency on specific data characteristics.**

In terms of limitations, this model is trained on a dataset of 60,000 images divided into 50,000 training images and 10,000 test images. A 20% validation split further reduced the training set available for model fitting. The images themselves are 32 x 32 and categorized into 10 classes. Taken as a whole, this dataset is comparatively small relative to modern computer vision datasets in both number of images and resolution of images. The model would likely have difficulty generalizing with higher-resolution photos without further training and fine-tuning.

Additionally, the model still has difficulty accurately classifying dogs and cats, as explained in Part I.2. Frogs also showed higher recall than precision, showing that the model tends to predict the frog class more frequently, resulting in a higher false-positive rate. These issues show that the model struggles with generalization in animals with similar features.

As this application is trained solely on the CIFAR-10 dataset, it would be largely ineffective in real-world scenarios without the addition of industry-related data. Because of this, the final model could only serve as a foundation for experimenting with real-world applications.

The computational requirements of this model are significantly higher than those of the baseline CNN architecture. Training the final model took approximately 16 hours, with final testing taking approximately nine minutes. This is due to the additional complexity of the model and the trials required for Bayesian optimization.

The largest limitation to the model was revealed in CIFAR-10-C testing. The results on the final model showed significantly better robustness compared to previous models up until Severity Level 5, where

the mean accuracy fell below 65% to be on par with Version 2 results. For all severities, Version 4 had a 73.4% mean accuracy, compared to the model's 88.7% clean accuracy, resulting in an accuracy drop of 15.3%. This result shows that the model, while performing well, still has room for significant improvement in real-world generalization and robustness.

**L. Acknowledge sources, using APA-formatted in-text citations and references, for content that is quoted, paraphrased, or summarized.**

Please see the following pages for a full bibliography.

## Bibliography

- Geeks for Geeks. (2025a, July 23). *What is Label Smoothing?* Retrieved from Geeks for Geeks: <https://www.geeksforgeeks.org/machine-learning/what-is-label-smoothing/>
- Geeks for Geeks. (2025b, July 23). *How to Measure Overfitting when Train and Validation Sample is Small in Keras Model using R.* Retrieved from Geeks for Geeks: <https://www.geeksforgeeks.org/deep-learning/how-to-measure-overfitting-when-train-and-validation-sample-is-small-in-keras-model-using-r/>
- Geeks for Geeks. (2025c, July 23). *What is Batch Normalization in Deep Learning?* Retrieved from Geeks for Geeks: <https://www.geeksforgeeks.org/deep-learning/what-is-batch-normalization-in-deep-learning/>
- Geeks for Geeks. (2025d, July 12). *Dropout in Neural Networks.* Retrieved from Geeks for Geeks: <https://www.geeksforgeeks.org/machine-learning/dropout-in-neural-networks/>
- Geeks for Geeks. (2025e, November 17). *Softmax Activation Function in Neural Networks.* Retrieved from Geeks for Geeks: <https://www.geeksforgeeks.org/deep-learning/the-role-of-softmax-in-neural-networks-detailed-explanation-and-applications/>
- Geeks for Geeks. (2026, February 2). *Handling Imbalanced Data for Classification.* Retrieved from Geeks for Geeks: <https://www.geeksforgeeks.org/machine-learning/handling-imbalanced-data-for-classification/>
- Google. (2025a, August 28). *Datasets: Class-imbalanced datasets.* Retrieved from Google for Developers: <https://developers.google.com/machine-learning/crash-course/overfitting/imbalanced-datasets>
- Google. (2026b). *Classification: Accuracy, recall, precision, and related metrics.* Retrieved from Google for Developers: <https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall>
- Gupta, L. (2021, April 26). *Batch Normalization and ReLU for solving Vanishing Gradients.* Retrieved from Medium: <https://lava18.medium.com/how-batch-normalization-and-relu-solve-vanishing-gradients-3f1a8ace1c88>
- Keras. (2026a). *EarlyStopping.* Retrieved from Keras: [https://keras.io/api/callbacks/early\\_stopping/](https://keras.io/api/callbacks/early_stopping/)
- Keras. (2026b). *ReduceLROnPlateau.* Retrieved from Keras: [https://keras.io/api/callbacks/reduce\\_lr\\_on\\_plateau/](https://keras.io/api/callbacks/reduce_lr_on_plateau/)
- Keras. (2026c). *Dropout layer.* Retrieved from Keras: [https://keras.io/api/layers/regularization\\_layers/dropout/](https://keras.io/api/layers/regularization_layers/dropout/)
- Krishnamurthy, B. (2024, February 26). *An Introduction to the ReLU Activation Function.* Retrieved from Built In: <https://builtin.com/machine-learning/relu-activation-function>
- Lee, A. (2024, May 30). *Adam vs SGD: What are the optimizers in neural network and when do we use?* Retrieved from Medium: <https://medium.com/@pumadd1227/adam-vs-sgd-what-are-the-optimizers-in-neural-network-and-when-do-we-use-238478a0eaea>
- Maurício, J., Domingues, I., & Bernardino, J. (2023, April 28). *Comparing Vision Transformers and Convolutional Neural Networks for Image Classification: A Literature Review. Applied Sciences.*
- Nagpal, A. (2024, October 3). *L1 and L2 Regularization Methods, Explained.* Retrieved from Built In: <https://builtin.com/data-science/l2-regularization>
- Nanos, G. (2025, February 28). *Neural Networks: Pooling Layers.* Retrieved from Baeldung: <https://www.baeldung.com/cs/neural-networks-pooling-layers>

- Prabhu, V. u., & Birhane, A. (2021). Large Datasets: A Pyrrhic Win for Computer Vision? *2021 IEEE Winter Conference on Applications of Computer Vision*, 1536 - 1546.
- Scikit-learn. (2026). *f1\_score*. Retrieved from Scikit-learn: [https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1\\_score.html](https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html)
- Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. *ICLR 2015*, 1-14.
- Torralba, A., Fergus, R., & Freeman, B. (2020, June 29). *TinyImages*. Retrieved from Massachusetts Institute of Technology: <https://groups.csail.mit.edu/vision/TinyImages/>
- Vipin. (2024, September 2). *Understanding the VGG Architecture: A Detailed Breakdown*. Retrieved from Medium: <https://blog.devgenius.io/understanding-the-vgg-architecture-a-detailed-breakdown-b579da9801de>
- Zhao, X., Wang, L., Zhang, Y., Han, X., Deveci, M., & Parmar, M. (2024, March 23). A review of convolutional neural networks in computer vision. *Artificial Intelligence Review*, 99.